

CSE573 Project: G27 – P13 – Text2SQL: Natural Language to SQL via LLM-Modulo Framework

Abhinav Gorantla, Tejas Ajay Parse, Aryan Brijeshkumar Patel,
Rohitha Somuri, Nandana Vikraman, Kamal Teja Annamdasu
Arizona State University, Tempe, AZ, USA
{agorant2,tparse,apate294}@asu.edu
{rsomuri1,nvikrama,kannamd1}@asu.edu

Abstract—Translating natural language questions into executable SQL queries, commonly known as Text2SQL, remains a challenging problem due to the inherent gap between ambiguous human language and the rigid semantics of relational databases. Existing approaches struggle with complex schemas, cross-domain generalization, and practical concerns such as query safety, constraint validity, and unauthorized data access. While large language models (LLMs) have shown promise in semantic parsing tasks, single-shot Text2SQL generation often fails to reliably handle these challenges at scale.

This project presents a modular Text2SQL framework inspired by the LLM-Modulo paradigm, in which deterministic external verifiers assist the language model with schema understanding, query validation, and therefore, enable iterative refinement. A targeted metadata extractor performs schema linking to reduce irrelevant context, and four external verifiers: a Syntax Verifier, an LLM Judge & Destructive action checker and a Query Graph Verifier, provide structured feedback that drives iterative query correction. We evaluate the framework on the Spider benchmark against classical and LLM-based baselines including Bi-LSTM Seq2Seq, Flan-T5, and Qwen2.5. Our proposed method achieves an exact match accuracy of 24.53%, an execution accuracy of 35.49%, and a syntax validity rate of 98.45%, substantially outperforming all baselines, with the majority of gains occurring within the first two refinement iterations. By focusing on inference-time strategies rather than training large models from scratch, this work demonstrates that modular verification and targeted schema retrieval are effective and practical approaches to improving the reliability, safety, and accuracy of LLM-based Text2SQL systems.

Index Terms—SQL, Natural Language, LLMs, Seq2Seq, Query Translation, LLM-Modulo, Text2SQL

I. INTRODUCTION

Modern organizations store vast amounts of information in relational databases, yet accessing this data typically requires knowledge of Structured Query Language (SQL) [3]. For many users, particularly non-technical stakeholders, writing correct and efficient SQL queries is a significant barrier due to the language’s strict syntax, schema dependencies, and logical constraints [14], [15]. This dependency on technical expertise limits the accessibility of data-driven decision making across organizations.

Text2SQL addresses this problem by automatically translating natural language questions into executable SQL queries [9], [12], [17]. The core challenge lies in bridging the gap between human language, which is inherently ambiguous and context-dependent, and SQL, which demands

precise structure and explicit semantics. Real-world databases further compound this difficulty through complex schemas involving multiple tables, intricate foreign key relationships, and domain-specific constraints that require deep reasoning to navigate correctly.

Recent advances in large language models (LLMs) have demonstrated strong performance on a range of semantic parsing tasks, making them a natural candidate for Text2SQL generation [12]. However, single-shot LLM-based generation frequently produces syntactically invalid queries, violates schema constraints, or generates semantically incorrect results, particularly on complex cross-domain benchmarks [17]. Beyond correctness, practical deployments also demand safety guarantees: improperly generated queries may expose restricted data, bypass access controls, or execute destructive operations that violate data governance policies [1].

To address these limitations, this project adopts the LLM-Modulo framework [7], in which an LLM acts as a generative core while structured external modules handle validation and feedback. Our proposed pipeline consists of three key stages: a metadata extractor that performs targeted schema linking to provide the LLM with only query-relevant schema context; an LLM that generates candidate SQL queries; and a suite of four external verifiers that evaluate the generated queries across syntax, safety and constraints. Verifier feedback is passed back to the LLM iteratively, enabling the model to correct errors without any additional training.

We evaluate our framework on the Spider benchmark [17], a widely-used cross-domain Text2SQL dataset, comparing against classical and LLM-based baselines including a Bi-LSTM Seq2Seq model, Flan-T5, and Qwen2.5. Our results show that the proposed LLM-Modulo framework achieves an exact match accuracy of 24.53%, an execution accuracy of 35.49%, and a syntax validity rate of 98.45%, substantially outperforming all baselines, and that the majority of accuracy gains materialize within the first two refinement iterations.

The remainder of this project report is organized as follows. Section II formally defines the Text2SQL problem. Section III reviews relevant prior work. Section IV describes the proposed system architecture and algorithms. Section V describes the dataset used. Section VI presents the evaluation methodology and results. Section VII describes the UI interface. Section IX details team contributions, and Section X concludes the paper.

II. PROBLEM STATEMENT

The core problem addressed in this project is the automatic translation of natural language questions into syntactically correct and semantically accurate SQL queries that can be executed against a target relational database or "Text2SQL" [9], [12], [17].

Formally, given a natural language question q and a database schema $\mathcal{S}$ consisting of tables, columns, primary keys, and foreign key relationships, the goal is to produce a SQL query $\hat{q}$ such that executing $\hat{q}$ against the database returns the correct result set intended by the user. This problem is non-trivial for several interconnected reasons.

Language Gap. Natural language is inherently ambiguous and context-dependent, while SQL demands precise structure and explicit semantics. Users may omit details, use informal phrasing, or reference database concepts indirectly, making correct interpretation difficult [14], [15].

Schema Complexity. Real-world databases often contain multiple tables with intricate foreign key relationships and domain-specific constraints. Correctly mapping user intent to the appropriate tables, columns, joins, and conditions requires deep reasoning over both linguistic meaning and database structure [8], [17].

Security & Privacy. Improperly generated queries may inadvertently expose restricted data, bypass intended access controls, or execute destructive operations such as DROP or DELETE that violate data governance policies [1]. A practical Text2SQL system must therefore incorporate safety checks alongside correctness guarantees.

Broad Accessibility. Non-technical users cannot easily formulate correct SQL, creating a dependency on technical experts even for simple data retrieval tasks [14]. The ultimate goal is to make structured data systems accessible to a broader range of users.

This project specifically investigates the following research questions:

- 1) Does targeted schema retrieval improve SQL generation accuracy compared to full-schema prompting?
- 2) Can LLMs leverage structured external verifier feedback for iterative query refinement?
- 3) Which verifiers yield the highest accuracy gains per refinement iteration?

III. RELATED WORK

Text2SQL has been an active area of research for several decades, evolving from rule-based systems to neural sequence-to-sequence models and, more recently, to large language model-based approaches. In this section, we review the most relevant prior work across these paradigms.

A. Classical and Neural Approaches

Early Text2SQL systems relied on hand-crafted rules and grammar-based parsers to map natural language to SQL [3]. While interpretable, these approaches were brittle and did not generalize across domains. The introduction of neural sequence-to-sequence models marked a significant shift, with

bidirectional LSTM architectures [4] demonstrating that end-to-end supervised learning could produce reasonable SQL from natural language inputs. These models, implemented using toolkits such as DyNet [13], established the supervised learning paradigm as a standard baseline for Text2SQL evaluation. However, classical neural approaches struggle with cross-domain generalization and complex multi-table queries, motivating the need for more expressive architectures.

B. LLM-Based Approaches

The emergence of large pre-trained language models has significantly advanced Text2SQL performance. Instruction-finetuned encoder-decoder models such as Flan-T5 [2] have shown strong zero- and few-shot capabilities on structured prediction tasks including SQL generation. Code-specialized models such as Qwen2.5 [6] further benefit from pretraining on large corpora of programming languages, which aids SQL generation due to the structural similarity between code and SQL syntax. Despite these advances, single-shot LLM-based generation remains unreliable on complex benchmarks, frequently producing syntactically invalid or semantically incorrect queries [12].

C. State-of-the-Art Text2SQL Systems

Several recent systems have proposed more structured approaches to improve LLM-based Text2SQL. RESDSQL [10] decouples schema linking from SQL skeleton parsing using a ranking-enhanced encoder and a skeleton-aware decoder, achieving strong performance on Spider by separating the structural and semantic components of generation. DAIL-SQL [5] optimizes in-context learning through Code Representation Prompts and a DAIL selection strategy for choosing high-quality few-shot examples. MAC-SQL [16] introduces a multi-agent framework in which a Selector prunes the schema, a Decomposer applies chain-of-thought reasoning, and a Refiner corrects errors through execution feedback. CodeS [11] proposes open-source models ranging from 1B to 15B parameters with coarse-to-fine value retrieval using BM25 and longest common subsequence matching for large schemas. These systems represent the current state of the art and serve as comparators for situating the performance of our proposed approach.

D. LLM-Modulo Framework

Our work is most directly inspired by the LLM-Modulo framework proposed by [7], which argues that LLMs are best utilized as generative components within a larger system that includes external critics and verifiers. Rather than relying solely on the LLM's internal reasoning, the framework structures the generation process around iterative critique and refinement loops driven by formal external modules. This paradigm has been shown to be effective in planning and reasoning tasks, and we apply it here to the Text2SQL domain by designing domain-specific verifiers that provide structured feedback on syntax, safety, and constraints.

IV. SYSTEM ARCHITECTURE & ALGORITHMS

This section describes the proposed LLM-Modulo-based framework for Text2SQL, which decomposes the query generation task into modular components. Instead of relying on single-shot generation, the system integrates schema-aware context selection, large language model (LLM) generation, and structured external verification within an iterative refinement loop. This modular design is inspired by the LLM-Modulo paradigm [7] and enables improved accuracy, robustness, and safety compared to traditional approaches.

A. Overview of the Pipeline

Given a natural language query q and a database schema $\mathcal{S}$, the system generates an executable SQL query $\hat{y}$ through a multi-stage pipeline. The process begins with metadata extraction, followed by SQL generation using an LLM, and concludes with verification and refinement. The generated query is iteratively improved using feedback from external verifiers until a valid and semantically correct output is obtained or a stopping condition is reached [7].

B. Metadata Extractor (Schema Linking)

The metadata extractor performs schema linking to identify the subset of schema elements relevant to the input query. This step reduces the context provided to the LLM and improves generation quality by eliminating irrelevant tables and columns. The approach leverages lexical similarity between query tokens and schema elements, along with structural information such as foreign key relationships to preserve valid join paths. Similar schema-linking strategies have been shown to improve Text2SQL performance in prior work [5], [10]. The resulting filtered schema $\mathcal{S}'$ is used as input to the LLM, enabling more focused and efficient query generation.

C. LLM Core

The LLM serves as the generative component of the framework, producing candidate SQL queries conditioned on the natural language input and the extracted schema. In this work, we evaluate Flan-T5 [2] (both the base and a SQL fine-tuned variant) and Qwen2.5 [6]. The input prompt consists of the natural language query, the schema representation, and optionally feedback from previous iterations. At iteration t , the model generates a query $y^{(t)} = \text{LLM}(q, \mathcal{S}', \text{feedback}^{(t-1)})$. This formulation allows the model to iteratively refine its output based on structured feedback.

D. External Verifiers

External verifiers are designed to evaluate the generated SQL query and provide structured feedback for refinement. These modules improve both correctness and safety by addressing limitations of LLM-based generation [7].

The *syntax verifier* checks whether the generated SQL query conforms to valid SQL grammar and identifies structural errors such as missing clauses or incorrect ordering. The *destructive action and authorization checker* ensures that the query does not perform unsafe operations, such as `DROP` or `DELETE`,

and enforces access control constraints to prevent unauthorized data retrieval [1].

The *constraint verifier* evaluates whether the generated query satisfies conditions implied by the natural language input, including filtering conditions, aggregations, and logical constraints. This step focuses on semantic correctness beyond syntactic validity.

E. Iterative Refinement Loop

The framework operates in an iterative loop where the generated SQL query is evaluated by all verifiers, and the resulting feedback is used to guide subsequent generations. If the query passes all verification checks, it is returned as the final output. Otherwise, the feedback is incorporated into the next iteration of the LLM. The process continues until a valid query is obtained or a predefined iteration limit is reached, following the iterative refinement paradigm proposed in prior work [7].

Empirical observations show that most improvements occur within the first one or two iterations, indicating that the refinement process is both effective and computationally efficient. Overall, the proposed architecture separates schema understanding, query generation, and verification into distinct components, enabling more reliable and interpretable Text2SQL performance.

V. DATASETS

To evaluate the proposed framework, we utilize the Spider benchmark, a widely adopted dataset for complex and cross-domain Text2SQL evaluation. The dataset provides a challenging setting for assessing a model's ability to generalize across diverse schemas and query structures.

A. Spider Dataset

The Spider dataset is a large-scale benchmark designed for evaluating complex and cross-domain Text2SQL systems [17]. It contains 10,181 natural language questions paired with 5,693 unique SQL queries across 200 relational databases spanning 138 domains. Each database includes multiple tables with non-trivial foreign key relationships, requiring models to perform reasoning over joins, aggregations, and nested queries.

A key characteristic of Spider is its database-split evaluation setting, in which test databases are entirely unseen during training. This prevents memorization of schema patterns and instead evaluates a model's ability to generalize across domains. Due to these properties, Spider serves as the primary dataset in this project for benchmarking accuracy, conducting ablation studies, and analyzing the impact of schema linking and verifier feedback.

B. Preprocessing and Usage

Prior to experimentation, the dataset undergoes preprocessing to ensure compatibility with the proposed pipeline. Database schemas are parsed to extract tables, columns, primary keys, and foreign key relationships, and are converted into a normalized textual representation suitable for

LLM prompting. Natural language queries are tokenized, and schema linking preparation is performed to enable efficient metadata extraction, following standard preprocessing practices in Text2SQL systems [12].

Since this work focuses on inference-time improvements rather than model training, the Spider dataset is primarily used for evaluation and analysis. It is used to benchmark baseline performance, conduct ablation studies, and evaluate the effectiveness of schema linking and iterative verifier feedback.

VI. EVALUATION

This section evaluates the proposed LLM-Modulo-based Text2SQL framework on the Spider benchmark [17]. The goal is to measure not only whether the system can generate correct SQL, but also whether targeted schema retrieval and external verifier feedback improve reliability, safety, and iterative refinement. We compare the proposed method against classical and LLM-based baselines under the same evaluation setting, and we conduct ablation studies to isolate the contribution of schema linking, verifier feedback, and refinement iterations [7], [12].

A. Evaluation Metrics

We evaluate the system using three metrics. **Exact Match (EM)** measures whether the generated SQL exactly matches the reference query after normalization, and therefore captures structural correctness. **Execution Accuracy (EX)** measures whether the generated SQL produces the same result set as the ground-truth query when executed on the target database, capturing semantic equivalence even when the generated query is not textually identical [17]. **Syntax Validity Rate (SVR)** measures the percentage of outputs that are syntactically valid SQL queries and can be executed without parsing errors.

B. Baselines

To assess the effectiveness of the proposed approach, we compare it against both classical and LLM-based baselines. The classical baseline is a **Bi-LSTM Seq2Seq** model, implemented using the DyNet toolkit and following the conventional supervised text-to-SQL formulation [4], [13]. For LLM comparison, we evaluate **Flan-T5 Base**, **Qwen2.5-0.5B**, and **Flan-T5 (SQL Fine-Tuned)** under a standard single-shot generation setting [2], [6]. The proposed **LLM-Modulo** system uses schema linking and iterative verifier feedback within the same evaluation protocol [7].

Table I shows that the proposed LLM-Modulo framework substantially outperforms all baselines on Spider. In particular, the proposed system reaches an EM of 24.53%, while the strongest baseline reported here is substantially lower. This improvement suggests that decomposing the task into schema retrieval, SQL generation, and external verification is more effective than single-shot generation alone [7], [10], [12].

A closer look at the baselines reveals several patterns worth highlighting. Flan-T5 Base produces essentially no valid SQL, with an SVR of only 2.22% and EM and EX of 0%, confirming that instruction tuning alone, without exposure to SQL

during training, is insufficient for this task and motivating the inclusion of a SQL fine-tuned variant. Qwen2.5-0.5B exhibits the opposite failure mode: a near-perfect syntax validity rate of 98.55% but an EM of only 1.06%, indicating that code-pretrained models reliably produce well-formed SQL but for the wrong query. This divergence between SVR and EM/EX shows that syntactic validity alone is an unreliable proxy for correctness. Notably, the classical Bi-LSTM Seq2Seq baseline outperforms both small LLM baselines on EM (4.74% versus 3.68% and 1.06%), despite a much lower SVR of 30.08%, suggesting that supervised training on Spider remains competitive with sub-billion-parameter LLMs in the absence of any external scaffolding.

Two further observations situate the gains of the proposed framework. First, the SVR of LLM-Modulo (98.45%) is essentially tied with that of the Qwen2.5-0.5B baseline (98.55%); the substantial gain over Qwen on EM (+23.47 points) and EX (+28.43 points) therefore does not arise from better syntactic well-formedness, but rather from the semantic correction driven by the constraints verifiers. Second, the execution accuracy of the proposed system (35.49%) exceeds its exact match accuracy (24.53%) by nearly eleven points, indicating that a substantial fraction of generated queries are semantically equivalent to the reference but textually divergent. This is a known limitation of EM as a metric for SQL evaluation [17] and suggests that the system’s true correctness is higher than EM alone reflects.

C. Ablation Study

We perform ablation studies to understand which parts of the pipeline contribute most to performance. First, we compare **full-schema prompting** with **schema-linked prompting**. The schema-linked setting reduces irrelevant context and allows the model to focus on the tables and columns most relevant to the natural language query. Second, we compare a **verifier-free pipeline** with versions that incrementally add the syntax verifier, destructive action and authorization checker, and constraint verifier. Third, we vary the number of refinement iterations to study how much improvement is obtained from repeated feedback cycles [7], [10].

Figure 1 summarizes the exact-match comparison across methods, while Figure 2 shows how verifier feedback changes performance across refinement iterations.

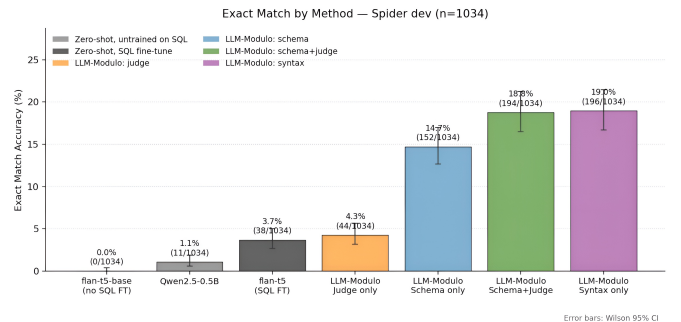


Fig. 1: Exact Match by Method on Spider.

TABLE I: Comparison of baseline methods and proposed approach on Spider.

Method	Type	EM (%)	EX (%)	SVR (%)
Bi-LSTM Seq2Seq	Classical ML	4.74	7.93	30.08
Flan-T5 Base	LLM Baseline	0.00	0.00	2.22
Qwen2.5-0.5B	LLM Baseline	1.06	7.06	98.55
Flan-T5 (SQL Fine-Tuned)	LLM Baseline	3.68	6.38	86.07
LLM-Modulo (Ours)	Proposed	24.53	35.49	98.45

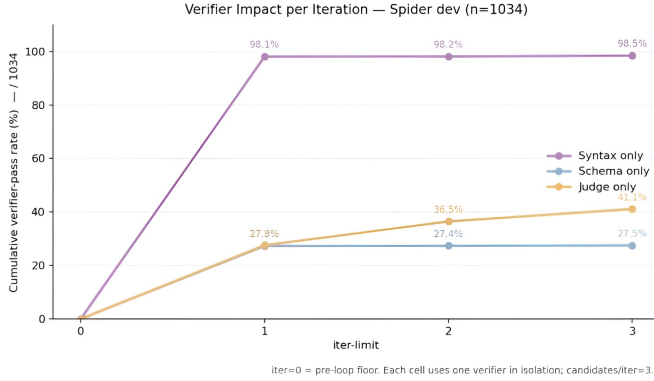


Fig. 2: Verifier impact per iteration.

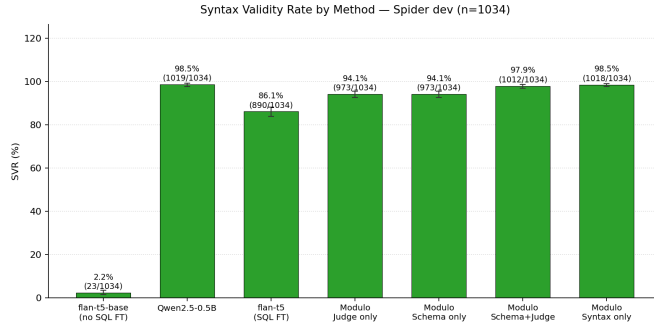


Fig. 4: Syntax Validity Rate across the methods we evaluated.

The main trend observed in our experiments is that targeted schema retrieval improves generation quality by reducing context noise, while external verifiers further improve correctness by catching syntactic and semantic errors before final output. Among the verifiers, the syntax verifier provides immediate benefit in early iterations, while the constraint verifier contribute to deeper semantic correction. The safety verifier also plays a critical role by blocking destructive queries and unauthorized access attempts [1].

D. Findings

Overall, the evaluation supports three main findings. First, **schema linking** is valuable because it narrows the input context to the most relevant database metadata, which helps the LLM generate more focused SQL. Second, **iterative verifier feedback** significantly improves output quality, especially in the first one or two refinement rounds.

Another important observation is that the majority of performance gain occurs early in the refinement process. This

suggests that the framework reaches diminishing returns after a small number of iterations, making the approach practical from both a quality and efficiency standpoint. Taken together, these results show that the LLM-Modulo design is a strong fit for Text2SQL because it separates schema understanding, query generation, and verification into modular stages that can be improved independently [7], [17]. Figure 1 and Figure 2 support these conclusions, while Table I shows the baseline comparison.

VII. UI / VISUALIZATION INTERFACE DESIGN

The project includes a web-based Text2SQL interface designed as an interactive query builder. As shown in Figure 6, the UI uses a two-column layout: the left side collects user inputs, while the right side displays the generated SQL output. Users enter a natural-language question in the query panel and provide the database schema before submitting the request through the *Generate SQL* button. The interface also includes an optional LLM-Modulo verifier toggle, allowing users to switch between faster single-pass generation and a slower verification loop.

A key design feature is the schema insertion workflow. The system supports two schema input modes: users can directly paste SQL DDL statements into a schema editor, or upload a .sql file through the upload interface. This makes the tool usable for both quick experiments and larger database schemas stored in external files.

The UI also maintains a session-level query history after SQL generation. Each history item stores the natural-language request, the generated SQL, the generation mode, and the time of generation. Selecting a history item restores the original natural-language query and its SQL output, making it easier for users to compare previous translations or revisit earlier results during a session.

To improve readability of generated SQL, the output panel renders SQL with syntax highlighting. SQL keywords such as SELECT, FROM, WHERE, JOIN, and aggregation functions are visually distinguished from strings, numbers, and comments. This makes generated queries easier to inspect, debug, and discuss in an experimental setting.

VIII. GRAPH-BASED VERIFIER

In addition to the primary verification pipeline, we explored a graph-based verifier as a complementary approach. The motivation behind this direction was to leverage the structural relationships inherent in the schema by representing it as a graph, where entities and their interactions could be modeled more explicitly. Mapping the existing schema into a graph

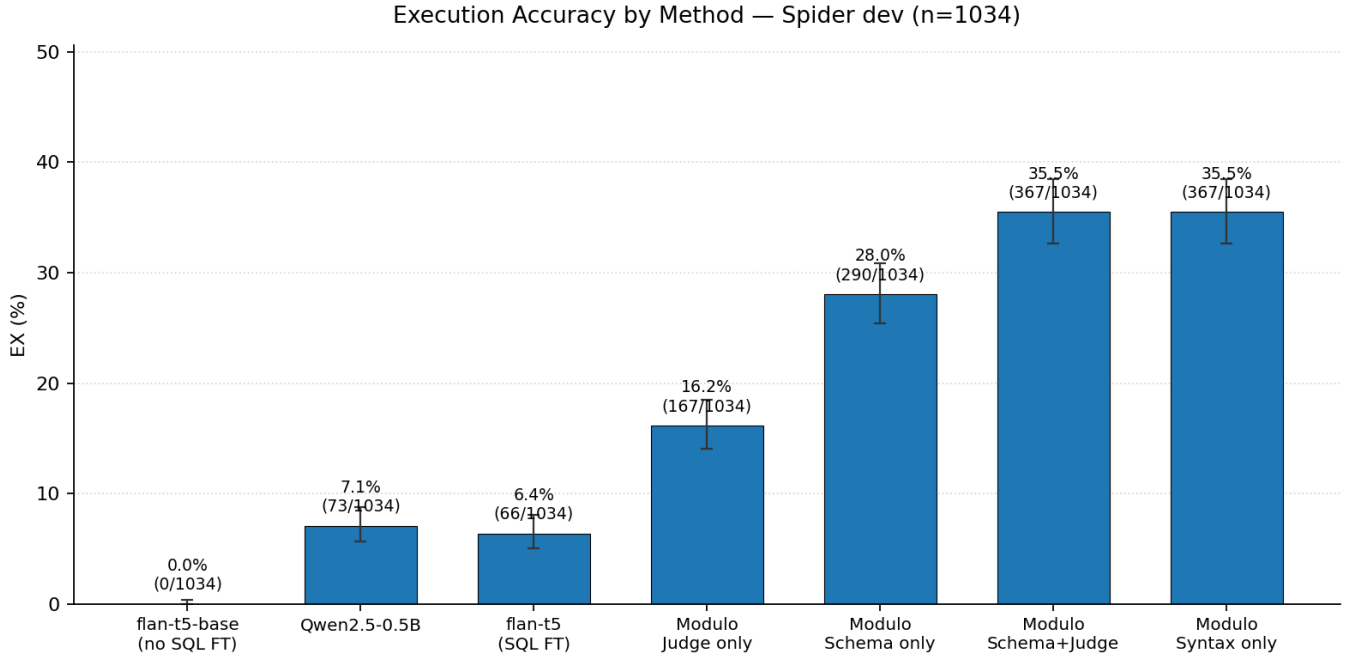


Fig. 3: Execution Accuracy across the methods we evaluated.

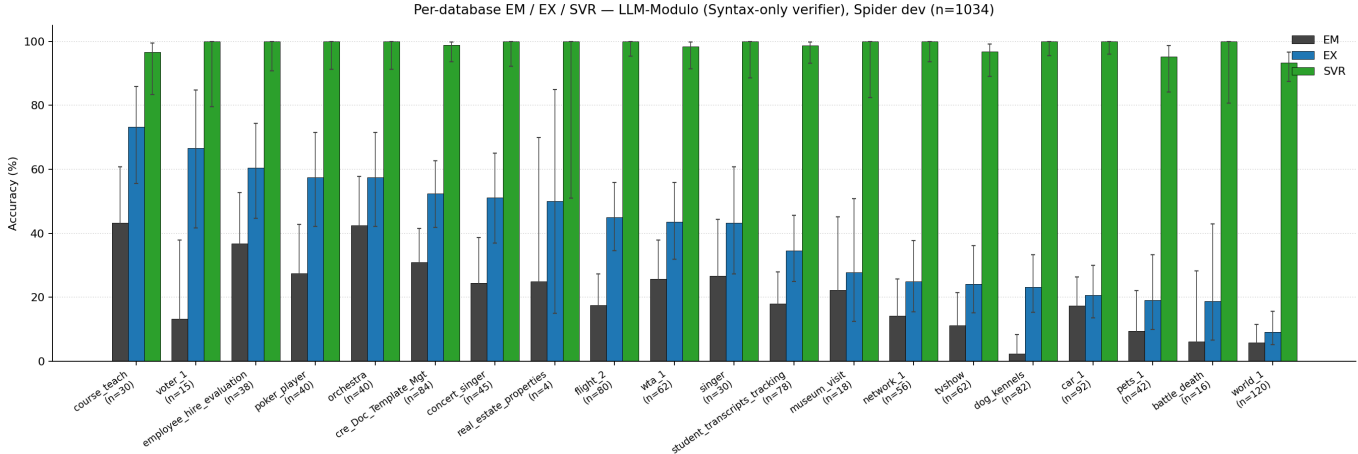


Fig. 5: Exact Match (EM), Execution Accuracy (EX) and Syntax Validity Rate (SVR) across databases in Spider (Sec. V-A).

representation proved to be relatively straightforward, as the relational structure naturally translated into nodes and edges.

However, the development of the graph-based verification algorithm presented significant challenges. In particular, ensuring correctness and termination of the traversal and validation logic was non-trivial.

Despite these limitations, the preliminary work indicates that graph-based verification remains a promising direction for future research. With further refinement of the underlying algorithms, particularly around termination guarantees and traversal constraints, this approach could offer a more expressive and robust framework for verification tasks.

IX. DIVISION OF WORK & TEAM CONTRIBUTIONS

The project was divided among five team members, with each member taking ownership of specific components while contributing to shared tasks such as testing, evaluation, and presentation preparation.

- **Abhinav Gorantla** was responsible for setting up the development environment, configuring and running Flan-T5, building the metadata extraction module, implementing the external verifiers, and integrating all components into the full LLM-Modulo pipeline and worked on the project report.
- **Tejas Ajay Parse** developed the core LLM-Modulo framework and was responsible for integrating the exter-

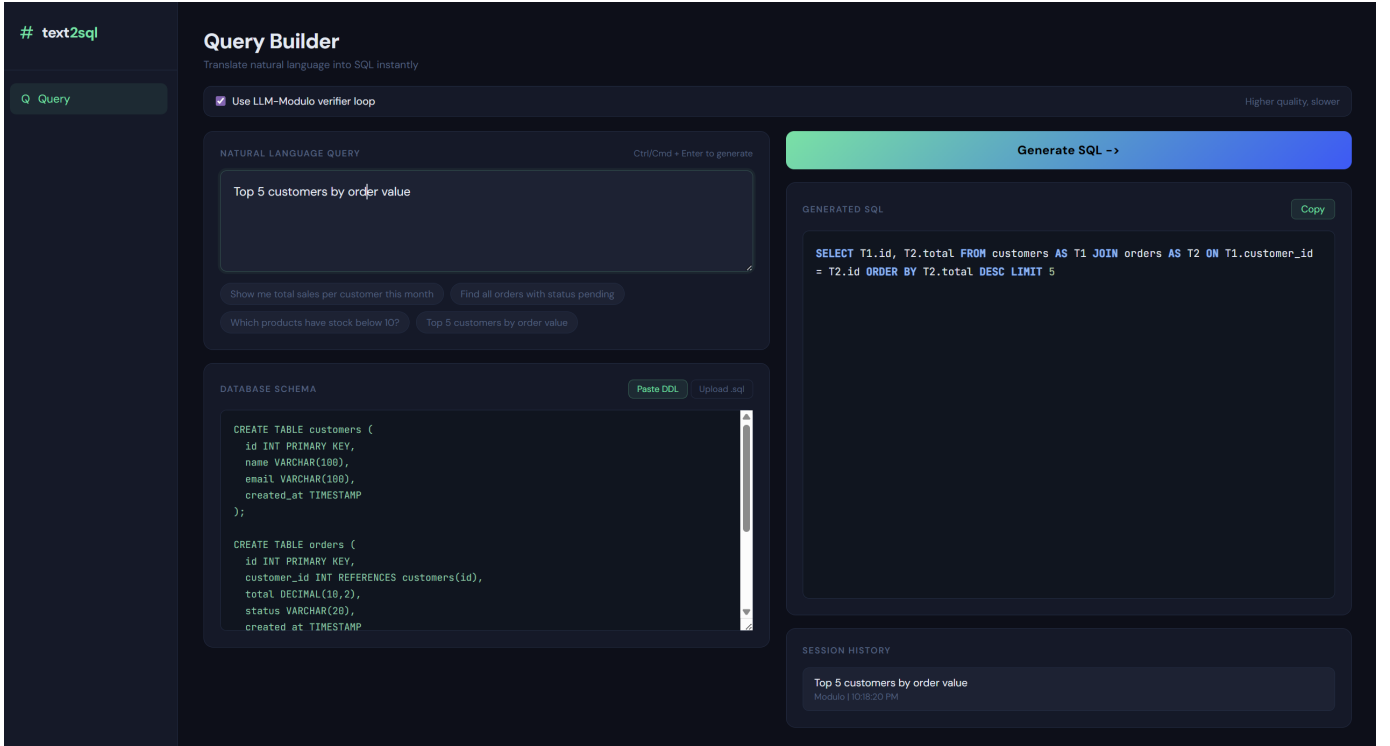


Fig. 6: Web-based Text2SQL query builder interface. The left panel accepts a natural-language query and database schema through pasted DDL or .sql upload, while the right panel displays generated SQL with syntax highlighting. The interface also supports session-level query history for revisiting previous natural-language and SQL pairs.

nal verifier pipeline into the overall system architecture.

- **Rohitha Somuri** managed dataset setup, implemented and evaluated the LLM baselines, conducted experimental testing across the evaluation configurations and worked on the project report.
- **Aryan Brijeshkumar Patel** designed and implemented the frontend user interface, was responsible for integrating the external verifiers with the UI layer and worked on the project report.
- **Nandana Vikraman** managed dataset setup, implemented and evaluated the LLM baselines, implemented the traditional machine learning models, contributed to experimental evaluation and worked on the project report.
- **Kamal Teja Annamdasu** worked on the project report.

X. CONCLUSIONS

In this work, we presented a modular Text2SQL framework based on the LLM-Modulo paradigm, aimed at improving the reliability and accuracy of translating natural language queries into executable SQL. Instead of relying on zero-shot generation, our approach decomposes the problem into three key components: targeted schema retrieval through a metadata extractor, SQL generation using a large language model, and iterative refinement using structured external verifiers.

Our experimental evaluation on the Spider benchmark demonstrates that this modular design leads to significant performance improvements over both classical and LLM-based

baselines. In particular, the proposed framework achieves an exact match accuracy of 24.53%, an execution accuracy of 35.49%, and a syntax validity rate of 98.45%, substantially outperforming all baseline methods considered. The results further show that the majority of performance gains are obtained within the first one or two refinement iterations, indicating that iterative verifier feedback is both effective and computationally efficient.

A key contribution of this work is the integration of external verification modules into the generation loop. The syntax verifier ensures structural correctness in early iterations, while the constraint and query graph verifiers contribute to improving semantic correctness in later stages. In addition, schema linking through targeted metadata extraction plays a critical role in reducing irrelevant context and enabling the model to focus on query-relevant tables and relationships. Together, these components demonstrate that combining LLMs with structured reasoning modules is a practical and effective strategy for Text2SQL tasks.

Despite these improvements, several limitations remain. First, the overall accuracy is still relatively low compared to human-level performance, highlighting the inherent difficulty of complex cross-domain Text2SQL tasks. Second, the framework introduces additional computational overhead due to multiple verification and refinement steps. Third, the effectiveness of the system depends on the quality of schema linking and verifier design, which may require domain-specific

tuning for different databases.

Future work can explore several directions to address these limitations. One promising direction is to incorporate more advanced schema linking techniques, such as graph-based retrieval methods, to further improve context selection. Another direction is to design more sophisticated verifier modules, including semantic or execution-based feedback mechanisms, to better capture logical correctness. Additionally, optimizing the refinement loop to reduce latency while maintaining performance would improve the practicality of deployment in real-world systems. Finally, extending the framework to handle large-scale and noisy enterprise databases, such as those found in the BIRD benchmark, would further validate its robustness and scalability.

Overall, this work demonstrates that the LLM-Modulo framework provides a strong foundation for building more reliable and interpretable Text2SQL systems by effectively combining the generative capabilities of LLMs with structured external reasoning components.

REFERENCES

- [1] CHAFIK, S., EZZINI, S., AND BERRADA, I. Enhancing security in text-to-sql systems: A novel dataset and agent-based framework. *Natural Language Processing* 31, 6 (2025), 1399–1422.
- [2] CHUNG, H. W., HOU, L., LONGPRE, S., ZOPH, B., TAY, Y., FEDUS, W., LI, Y., WANG, X., DEGHANI, M., BRAHMA, S., WEBSON, A., GU, S. S., DAI, Z., SUZGUN, M., CHEN, X., CHOWDHERY, A., CASTRO-ROS, A., PELLAT, M., ROBINSON, K., VALTER, D., NARANG, S., MISHRA, G., YU, A., ZHAO, V., HUANG, Y., DAI, A., YU, H., PETROV, S., CHI, E. H., DEAN, J., DEVLIN, J., ROBERTS, A., ZHOU, D., LE, Q. V., AND WEI, J. Scaling instruction-finetuned language models, 2022.
- [3] CODD, E. F. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (June 1970), 377–387.
- [4] FINEGAN-DOLLAH, C., KUMMERFELD, J. K., ZHANG, L., RAMANATHAN, K., SADASIVAM, S., ZHANG, R., AND RADEV, D. Improving text-to-sql evaluation methodology. *arXiv preprint arXiv:1806.09029* (2018).
- [5] GAO, D., WANG, H., LI, Y., SUN, X., QIAN, Y., DING, B., AND ZHOU, J. Text-to-sql empowered by large language models: A benchmark evaluation. In *Proceedings of the VLDB Endowment* (2023).
- [6] HUI, B., YANG, J., CUI, Z., YANG, J., LIU, D., ZHANG, L., LIU, T., ZHANG, J., YU, B., LU, K., DANG, K., FAN, Y., ZHANG, Y., YANG, A., MEN, R., HUANG, F., ZHENG, B., MIAO, Y., QUAN, S., FENG, Y., REN, X., REN, X., ZHOU, J., AND LIN, J. Qwen2.5-coder technical report, 2024.
- [7] KAMBHAMPATI, S., VALMEEKAM, K., GUAN, L., VERMA, M., STECHLY, K., BHAMBRI, S., SALDY, L., AND MURTHY, A. LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks, June 2024. *arXiv:2402.01817* [cs].
- [8] KATSOGIANNIS-MEIMARAKIS, G., AND KOUTRIKA, G. A survey on deep learning approaches for text-to-SQL. *The VLDB Journal* 32, 4 (July 2023), 905–936.
- [9] KATSOGIANNIS-MEIMARAKIS, G., XYDAS, M., AND KOUTRIKA, G. Natural language interfaces for databases with deep learning. *Proc. VLDB Endow.* 16, 12 (2023), 3878–3881.
- [10] LI, H., ZHANG, J., LI, C., AND CHEN, H. Resdsq: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2023).
- [11] LI, H., ZHAO, J., LI, C., AND CHEN, H. Codes: Towards building open-source language models for text-to-sql. In *Proceedings of the ACM on Management of Data* (2024).
- [12] LIU, X., SHEN, S., LI, B., MA, P., JIANG, R., ZHANG, Y., FAN, J., LI, G., TANG, N., AND LUO, Y. A survey of text-to-sql in the era of llms: Where are we, and where are we going?, 2025.
- [13] NEUBIG, G., DYER, C., GOLDBERG, Y., MATTHEWS, A., AMMAR, W., ANASTASOPOULOS, A., BALLESTEROS, M., CHIANG, D., CLOTHIAUX, D., COHN, T., ET AL. Dynet: The dynamic neural network toolkit. *arXiv preprint arXiv:1701.03980* (2017).
- [14] SHIN, S.-S. Effect of semantic distance on learning structured query language: An empirical study. *Frontiers in Psychology Volume 13 - 2022* (2022).
- [15] TAIPALUS, T. The effects of database complexity on sql query formulation. *Journal of Systems and Software* 165 (2020), 110576.
- [16] WANG, B., REN, C., YANG, J., LIANG, X., BAI, J., CHAI, L., YAN, Z., ZHANG, Q.-W., YIN, D., SUN, X., ET AL. Mac-sql: A multi-agent collaborative framework for text-to-sql. In *arXiv preprint arXiv:2312.11242* (2024).
- [17] YU, T., ZHANG, R., YANG, K., YASUNAGA, M., WANG, D., LI, Z., MA, J., LI, I., YAO, Q., ROMAN, S., ZHANG, Z., AND RADEV, D. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (Brussels, Belgium, Oct. 2018), E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds., Association for Computational Linguistics, pp. 3911–3921.